

UNIVERSIDADE VILA VELHA

Curso de Ciência da Computação

Atividade Avaliativa — Contexto "UVV Steam"

UML, GUI e Jogos — Desenvolvimento orientado a objetos em C#

Jogos escolhidos:

Jogo 03 — Football Penalty Shootout Game

Jogo 17 — Car Racing Game (WPF)

Integrantes do grupo:

Isaque Rocon

Ricardo Tamanini

Antonio

João Pedro

Sumário

1. Descritivo do projeto
2. Jogo 03 — Football Penalty Shootout Game
 - 2.1 Diagrama de casos de uso
 - 2.2 Diagrama de classes
 - 2.3 Novas funcionalidades implementadas
3. Jogo 17 — Car Racing Game (WPF)
 - 3.1 Diagrama de casos de uso
 - 3.2 Diagrama de classes
 - 3.3 Novas funcionalidades implementadas
4. Planejamento das atividades
5. Uso de inteligência artificial
6. Conclusão

1. Descritivo do projeto

Esta atividade faz parte do contexto "UVV Steam", um catálogo de jogos desenvolvidos em C# a partir do qual cada grupo deveria escolher dois jogos, compreender seu funcionamento e realizar contribuições obrigatórias. O grupo escolheu os seguintes jogos:

- **Jogo 03 — Football Penalty Shootout Game:** o jogador escolhe a direção do chute clicando em uma região do gol; um goleiro controlado por IA simples tenta defender, e o placar registra gols e erros.

Jogo 17 — Car Racing Game (WPF): o jogador controla um carro que precisa desviar do tráfego, usando as setas do teclado; estrelas coletáveis ativam um "modo de poder" temporário que permite atravessar os carros adversários sem perder o jogo.

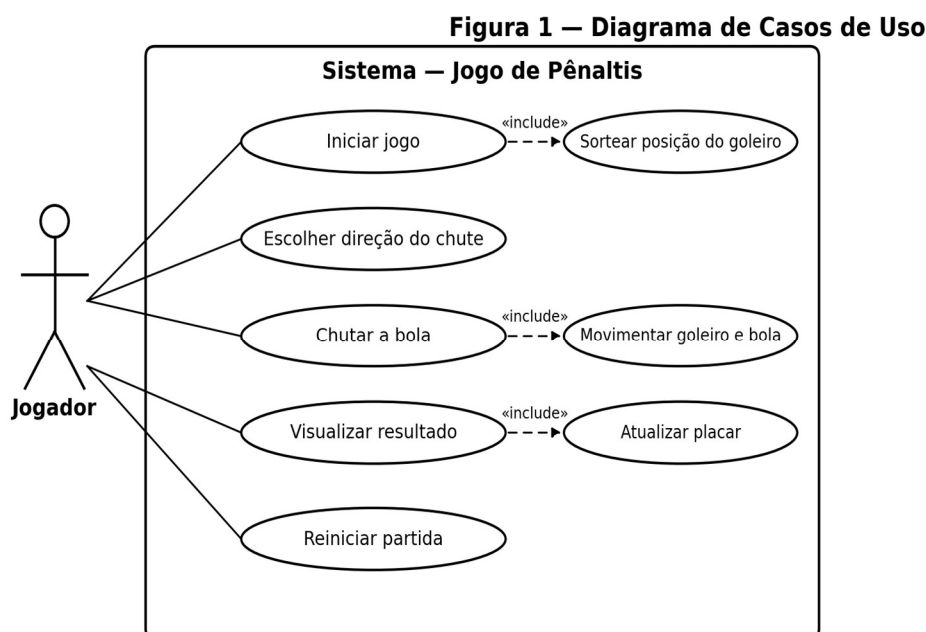
Para cada jogo foram produzidos, com apoio de recursos de UML, o diagrama de casos de uso e o diagrama de classes, e foram implementadas novas funcionalidades obrigatórias, documentadas com evidência de código antes/depois nas seções 2.3 e 3.3.

2. Jogo 03 — Football Penalty Shootout Game

Jogo original em Windows Forms (C#) no qual o jogador cobra pênaltis contra um goleiro controlado pelo computador. A cada clique em uma das cinco regiões do gol (esquerda, direita, cima, cima-esquerda, cima-direita) a bola é animada até o alvo e o goleiro salta para uma posição sorteada com base em um nível de dificuldade.

2.1 Diagrama de casos de uso

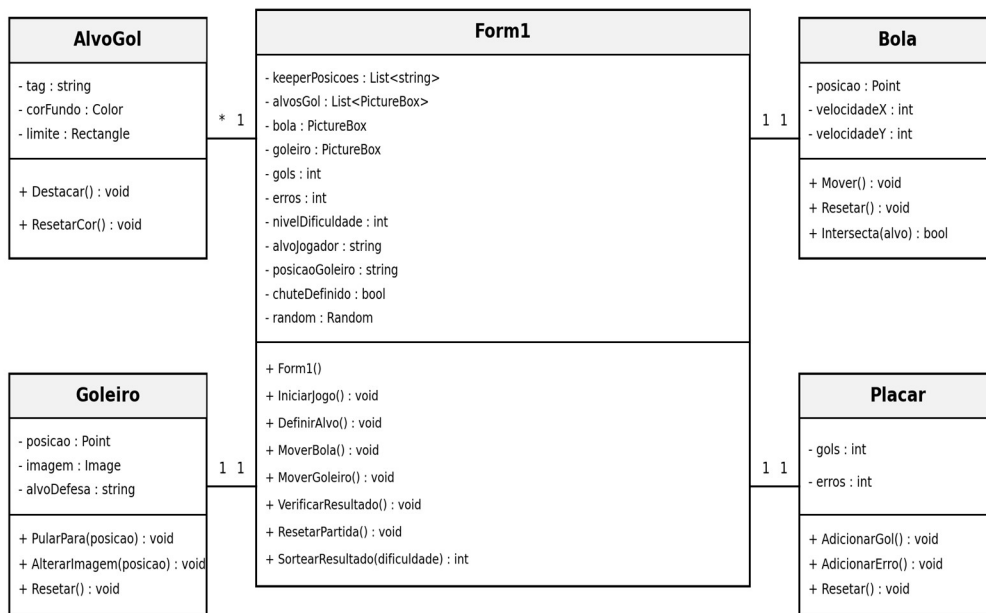
O ator principal é o Jogador. As ações de calcular a posição de defesa do goleiro e verificar o resultado do chute são incluídas («include») automaticamente no fluxo de escolha da direção do chute, já que ocorrem sem interação direta adicional do usuário.



2.2 Diagrama de classes

O código original concentra toda a lógica na classe Form1. Para a modelagem UML, essa responsabilidade foi decomposta em classes de domínio coesas — Form1 (controladora), GoalTarget,

GoalKeeper, Ball e ScoreBoard — associadas por composição e associação, já que nenhuma delas existe fora do ciclo de vida de uma partida.



2.3 Novas funcionalidades implementadas

Foram implementadas duas contribuições obrigatórias sobre o código original.

2.3.1 Botão de reiniciar partida funcional

O evento de clique do botão de reset já existia como estrutura, mas estava totalmente comentado — ou seja, o placar nunca era zerado na interface. A funcionalidade foi ativada e conectada corretamente ao método ResetGame() já existente.

2.3.2 Sistema de dificuldade e sorteio do resultado

A nova regra de dificuldade define um limite diferente para o número aleatório gerado pelo sistema. Em cada nível, o sorteio é realizado dentro de uma faixa específica e, em seguida, uma estrutura de seleção compara o número sorteado com 0 para determinar se o jogador perde ou acerta o chute.

1. Na dificuldade 1, o sistema gera um número aleatório entre 0 e 5.
2. Na dificuldade 2, o sistema gera um número aleatório entre 0 e 4.
3. Na dificuldade 3, o sistema gera um número aleatório entre 0 e 3.
4. Depois do sorteio, uma estrutura de seleção analisa o número gerado e o compara com 0.
5. Se número == 0, o jogador perde a tentativa; caso contrário, o jogador acerta.

Fluxo da decisão:

```
int numero = SortearResultado(nivelDificuldade);
```

```
if (numero == 0)
```

```
    miss++;
else
    goal++;
```

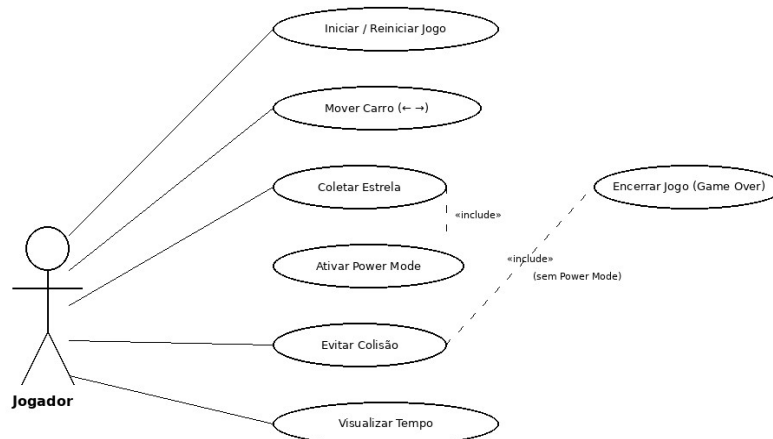
Essa regra torna as dificuldades diferentes entre si: quanto maior o nível, menor é a faixa de sorteio e, consequentemente, maior é a chance de o número sorteado ser 0.

3. Jogo 17 — Car Racing Game (WPF)

Jogo em WPF no qual o jogador controla um carro (setas esquerda/direita) para desviar do tráfego que desce pela pista. Estrelas aparecem periodicamente; ao serem coletadas, ativam um "modo de poder" que permite atravessar os carros adversários por um tempo limitado sem encerrar o jogo.

3.1 Diagrama de casos de uso

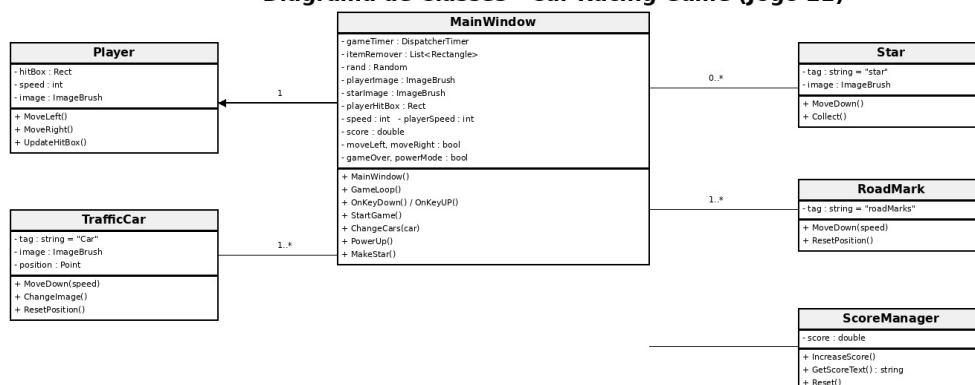
O ator principal também é o Jogador. "Coletar estrela" inclui («include») o caso de uso "Ativar modo de poder", pois toda coleta bem-sucedida dispara automaticamente esse comportamento. A colisão sem Power Mode inclui o encerramento do jogo.



3.2 Diagrama de classes

A classe MainWindow do código original concentra o game loop inteiro. Na modelagem, essa responsabilidade foi separada em MainWindow (controladora), Player, TrafficCar, Star, RoadMark e ScoreManager — as classes de objetos da pista com multiplicidade "muitos", refletindo o uso de uma coleção de Rectangle diferenciados por Tag no código.

Diagrama de Classes - Car Racing Game (Jogo 21)



3.3 Novas funcionalidades implementadas

Foram implementadas duas contribuições obrigatórias sobre o código original.

3.3.1 Pausar e retomar a partida (tecla P)

O jogo original não tinha nenhuma forma de pausa — uma vez iniciado, o loop rodava ininterruptamente até a colisão. Foi adicionado o controle de pausa pela tecla P, interrompendo a lógica do GameLoop sem parar o DispatcherTimer, e exibindo uma mensagem indicativa na tela.

Antes:

```
private void OnKeyDown(object sender, KeyEventArgs e)
{
    if (e.Key == Key.Left) { moveLeft = true; }
    if (e.Key == Key.Right) { moveRight = true; }
}
private void GameLoop(object sender, EventArgs e)
{
    score += .05;
    ...
}
```

Depois:

```
bool isPaused = false; // NOVA FUNCIONALIDADE: pausa
private void OnKeyDown(object sender, KeyEventArgs e)
{
    if (e.Key == Key.Left) { moveLeft = true; }
    if (e.Key == Key.Right) { moveRight = true; }
    if (e.Key == Key.P && gameOver == false)
    {
        isPaused = !isPaused;
        scoreText.Content = isPaused
            ? "Pausado - pressione P para continuar"
            : "Survived " + score.ToString("#.#") + " Seconds";
    }
}
private void GameLoop(object sender, EventArgs e)
{
    if (isPaused) return; // interrompe a lógica sem parar o timer
    score += .05;
    ...
}
```

3.3.2 Recorde de sobrevivência na sessão

O placar mostrava apenas a pontuação da rodada atual, sem nenhum histórico. Foi adicionado o rastreamento do melhor tempo de sobrevivência dentro da sessão de jogo (persistindo entre replays, até fechar o programa), exibido junto à mensagem de fim de jogo.

Antes:

```
gameTimer.Stop();
scoreText.Content += " Press Enter to replay";
gameOver = true;
```

Depois:

```
double bestScore = 0; // NOVA FUNCIONALIDADE: recorde da sessão
...
gameTimer.Stop();
if (score > bestScore) bestScore = score;
scoreText.Content += $" Press Enter to replay | Melhor: {bestScore:0.0}s";
gameOver = true;
```

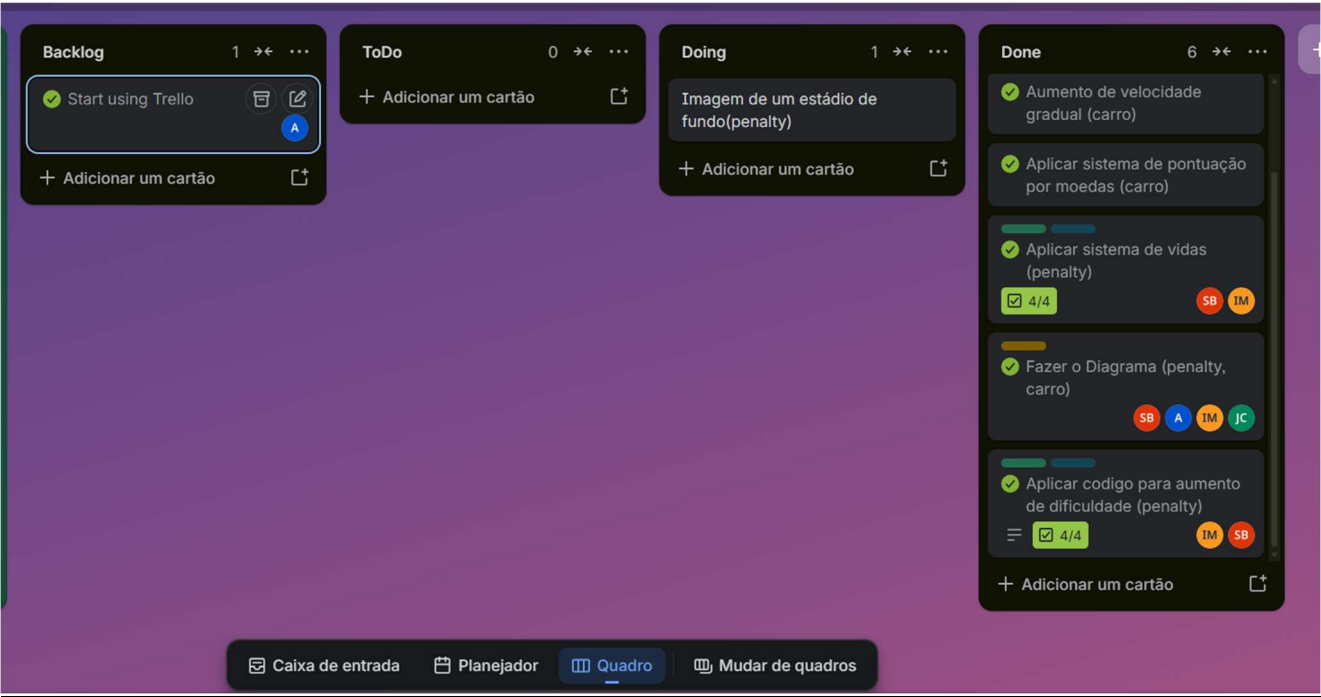
Reflexão: por ser um campo de instância inicializado uma única vez, bestScore não é reiniciado em StartGame(), preservando o recorde entre partidas. Se o requisito fosse persistir entre execuções do programa, seria necessário gravar esse valor em arquivo ou registro, o que ficou fora do escopo desta contribuição.

4. Planejamento das atividades

O planejamento individual foi acompanhado em sala de aula por meio de um quadro Kanban (Backlog / To Do / Doing / Done) na ferramenta Trello. A distribuição de responsabilidades entre os integrantes foi:

Tarefa	Responsável	Status
Levantamento do código-fonte e escolha dos jogos	Todo o grupo	Concluído
Diagrama de casos de uso e de classes — Jogo 03 (pênaltis)	Isaque Rocon, Ricardo Tamanini	Concluído
Diagrama de casos de uso e de classes — Jogo 17 (corrida)	João Pedro, Antonio	Concluído
Implementação das novas funcionalidades — Jogo 03	Isaque Rocon, Ricardo Tamanini	Concluído
Implementação das novas funcionalidades — Jogo 17	João Pedro, Antonio	Concluído
Testes manuais e validação das contribuições	Todo o grupo	Concluído
Consolidação da documentação e revisão final	Todo o grupo	Concluído

Evidência do quadro Kanban utilizado no Trello durante o desenvolvimento:



Observação: o planejamento completo, com datas e movimentação das tarefas entre colunas, está registrado na ferramenta de gestão de projetos utilizada pelo grupo (Trello), acompanhada semanalmente em sala de aula conforme solicitado no enunciado.

5. Uso de inteligência artificial

Durante o desenvolvimento do trabalho, o grupo utilizou inteligência artificial como ferramenta de apoio, principalmente para compreender os códigos dos jogos, esclarecer dúvidas e auxiliar em algumas etapas da documentação.

5.1 Ferramentas e prompts utilizados

Ferramenta: assistente de linguagem (Claude), via interface de chat.

- **Prompt:** solicitações para explicar partes do código que não estavam claras, compreender o funcionamento de determinados métodos e estruturas e esclarecer dúvidas que surgiram durante o desenvolvimento.
- **Prompt:** busca de explicações sobre comandos e recursos do C# que precisávamos utilizar nos projetos.
- **Prompt:** auxílio na compreensão e organização dos diagramas de casos de uso e de classes.

5.2 Critérios de revisão humana

- As explicações e sugestões fornecidas pela IA foram analisadas pelos integrantes e conferidas com o código dos projetos antes de serem utilizadas.
- As funcionalidades implementadas foram executadas e testadas manualmente no Visual Studio para verificar seu funcionamento.

5.3 Nível empregado na AI

A IA foi utilizada como apoio ao desenvolvimento e aprendizado, principalmente para compreender códigos, esclarecer dúvidas, pesquisar comandos e auxiliar na organização da documentação e dos diagramas.

As decisões sobre as alterações, a implementação no código, os testes e a revisão final foram realizadas pelos integrantes do grupo.

5.4 Evidências comparativas

As alterações realizadas nos jogos estão apresentadas nas seções 2.3 (Jogo 03) e 3.3 (Jogo de corrida), mostrando as funcionalidades desenvolvidas e as diferenças entre o código original e as modificações realizadas pelo grupo.

As tarefas de instalação e configuração dos projetos, execução dos jogos, testes das funcionalidades e revisão final do código foram realizadas manualmente pelos integrantes.

6. Conclusão

Durante o desenvolvimento do trabalho, o grupo conseguiu entender melhor o funcionamento dos dois jogos escolhidos no catálogo “UVV Steam” e, a partir disso, elaborar os diagramas de UML e implementar as melhorias propostas. Também buscamos manter as alterações compatíveis com a estrutura original dos códigos em C#, realizando testes para verificar se as novas funcionalidades estavam funcionando corretamente.

A utilização de inteligência artificial ajudou em algumas etapas do trabalho, principalmente na organização das ideias, criação dos diagramas e apoio na documentação. Porém, todo o conteúdo foi revisado pelo grupo e as alterações no código foram analisadas e testadas antes de serem incluídas no projeto.

A divisão das tarefas entre os integrantes e o acompanhamento das atividades pelo Trello ajudaram na organização do desenvolvimento. As funcionalidades implementadas também foram registradas com exemplos de antes e depois, facilitando a comparação e mostrando o que foi alterado em cada jogo. Dessa forma, o grupo conseguiu concluir as etapas propostas e aplicar na prática os conhecimentos trabalhados durante a atividade.